Deterministic CSV Analytics QA Agent

A fully local, offline-capable analytics assistant that lets users upload CSV files and ask natural-language questions about their data. Built with **Streamlit**, **Pandas**, and **RapidFuzz** — no OpenAI APIs, no cloud LLMs, no internet required after installation.

Table of Contents

- What It Does
- Tech Stack
- How It Works
- Supported Queries
- Project Structure
- Setup & Deployment
- Column Mapping Guide
- Every Improvement Made
- Why Deterministic
- Limitations
- Future Roadmap

What It Does

Upload any CSV file and ask questions in plain English. The app converts your question into deterministic pandas operations using a rule-based engine.

Example Questions

Basic Analytics:

- `how many rows`
- `show columns`
- `describe dataset`

Aggregations:

- `average sales`
- `total profit`
- `highest revenue`
- `lowest employees`

Group-By Analysis:

- `profit by category`
- `sales by region`
- `average revenue by plan`

Time-Series Metrics (DAU/WAU/MAU):

- `dau last week`
- `wau this month`
- `mau past 30 days`

Cohort Retention Analysis:

- `which cohort has highest d7 retention`
- `lowest d7 retention cohort`
- `show retention summary`
- `d30 retention by cohort`

Tech Stack

Component	Technology	Why We Chose It
Frontend/UI	Streamlit	Fastest way to build interactive data apps with file upload, tables, and widgets
Analytics Engine	Pandas	Industry standard for CSV processing, grouping, filtering, and datetime operations
NLP Layer	RapidFuzz	Typo-tolerant fuzzy matching for column name detection without LLMs
Backend Logic	Python	Easy integration, deterministic rule engine, fully offline
Query Processing	Deterministic Rule Engine	Regex + if/else logic —same input always produces same output

---

How It Works

```
User Question → Normalize Text → Detect Intent → Match Columns → Run Pandas → Show Result
```

1. **Upload CSV** → Read into pandas DataFrame
2. **Ask Question** → Text is normalized (lowercase, strip spaces)
3. **Detect Intent** → Check against known patterns (DAU, retention, group-by, etc.)
4. **Match Columns** → Use fuzzy matching to find relevant columns
5. **Manual Override** → User can select columns via dropdowns if auto-detection fails
6. **Execute Query** → Run deterministic pandas operation
7. **Display Result** → Show as table, text, or downloadable CSV

Supported Queries

1. Basic Intents

Query	Result
`how many rows`	Total row count
`show columns`	List of all columns
`describe dataset`	Column names, dtypes, nulls, unique counts

2. Aggregation Queries

Query	Pandas Operation
`average sales`	`df['sales'].mean()`
`total profit`	`df['profit'].sum()`
`highest revenue`	`df['revenue'].max()`
`lowest employees`	`df['employees'].min()`

3. Group-By Queries

Query	Pandas Operation
`profit by category`	`df.groupby('category')['profit'].sum()`
`sales by region`	`df.groupby('region')['sales'].sum()`
`average revenue by plan`	`df.groupby('plan')['revenue'].mean()`

Pattern: `[metric] by [dimension]` — supports sum, mean, max, min based on keywords.

4. DAU / WAU / MAU

Query	What It Computes
`dau last week`	Distinct users per day, filtered to last 7 days
`wau this month`	Distinct users per week, filtered to current month
`mau past 30 days`	Distinct users per month, filtered to last 30 days

Requires: `user_id` column + date/timestamp column.

5. Cohort Retention Analysis

Query	What It Computes
`highest d7 retention cohort`	Cohort with best Day-7 retention rate
`lowest d7 retention cohort`	Cohort with worst Day-7 retention rate
`d7 retention by cohort`	Day-7 retention for all cohorts
`show retention summary`	D1, D3, D7, D14, D30 retention table

Requires: `user_id` + cohort date (e.g., `signup_date`) + activity date (e.g., `event_date`).

Retention Formula:

```
Retention Rate = (Users active on Day N / Total users in cohort) × 100
```

Project Structure

Option A: Single-File (Recommended for Streamlit Cloud)

```
deterministic-qa-agent/
├── app.py              # Everything in one file (UI + engine)
├── requirements.txt    # Dependencies
└── README.md           # This file
```

Option B: Two-File (Modular)

```
deterministic-qa-agent/
├── app.py              # Streamlit UI
├── query_engine.py     # Deterministic analytics engine
├── requirements.txt    # Dependencies
└── README.md           # This file
```

> **Note:** We switched to single-file to eliminate import errors on Streamlit Cloud.

Setup & Deployment

Local Setup

```bash
# 1. Clone repo
git clone https://github.com/yourusername/deterministic-qa-agent.git
cd deterministic-qa-agent

# 2. Install dependencies
pip install -r requirements.txt

# 3. Run the app
streamlit run app.py
```

Streamlit Cloud Deployment

1. Push code to GitHub
2. Go to share.streamlit.io
3. Connect your GitHub repo
4. Set main file path to `app.py`
5. Deploy

Requirements

```
streamlit
pandas
rapidfuzz
```

Column Mapping Guide

The app auto-detects columns, but you can manually override via dropdowns:

Dropdown	Purpose	Example Column Names
Date/timestamp	General date column for DAU/WAU/MAU	`date`, `timestamp`, `created_at`, `event_date`
Cohort date	When user first joined (for retention)	`signup_date`, `registration_date`, `first_seen`, `install_date`
User identifier	Unique user ID	`user_id`, `uid`, `customer_id`, `email`
Activity date	When user returned (for retention)	`event_date`, `login_date`, `session_date`, `purchase_date`

Important Notes

- **For single-date CSVs** (only one date column): Set both **Cohort date** and **Activity date** to the same column. The app will derive cohort date as each user's first date.
- **For two-date CSVs**: Set **Cohort date** to signup column and **Activity date** to event column.
- **If auto-detection fails**: Manually select from dropdowns.

Every Improvement Made

Improvement 1: Added `sys.path` Fix for Streamlit Cloud Imports

Problem: `ImportError: cannot import query_engine` on Streamlit Cloud. **Root Cause:** Streamlit Cloud runs from a different working directory. **Fix:** Added explicit `sys.path.insert(0, BASE_DIR)` before imports.

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
if BASE_DIR not in sys.path:
    sys.path.insert(0, BASE_DIR)
```

Improvement 2: Expanded User Column Detection

Problem: "I could not detect a user identifier column" for non-standard names. **Fix:** Expanded `USER_COL_HINTS` from 8 to 24 hints, including `anonymous_id`, `device_id`, `session_id`, `email`, `phone`.

Improvement 3: Added Manual Column Selection UI

Problem: Auto-detection fails on unusual column names like `actorId` or `createdOn`. **Fix:** Added `st.selectbox` dropdowns for all four column types with "Auto-detect" option.

Improvement 4: Added Generic Group-By Support

Problem: `profit by category` returned "Query not recognized". **Root Cause:** Engine only had hard-coded intents, no generic `X by Y` parser. **Fix:** Added `handle_by_query()` with regex pattern `([metric]) by ([group])` + `infer_numeric_column()` + `infer_group_column()`.

Improvement 5: Added Aggregation Function Detection

Problem: Could only do `sum` in group-by queries. **Fix:** Added `detect_agg_func()` that maps keywords to pandas operations:

- `average`, `avg` → `mean()`
- `highest`, `max` → `max()`
- `lowest`, `min` → `min()`
- `total`, `sum` → `sum()` (default)

Improvement 6: Added DAU/WAU/MAU Support

Problem: "what was dau last week?" was unanswerable. **Root Cause:** Engine lacked time-aware metric logic. **Fix:** Added:

- `METRIC_ALIASES` for DAU/WAU/MAU detection
- `TIME_PATTERNS` for relative date parsing (`last week`, `past 7 days`)
- `compute_dau()`, `compute_wau()`, `compute_mau()` with pandas datetime grouping
- `parse_time_window()` for date filtering

Improvement 7: Added Cohort Retention Analysis

Problem: "which cohort has highest d7 retention?" was unanswerable. **Root Cause:** Cohort analysis requires completely different logic from simple aggregations. **Fix:** Added:

- `detect_cohort_query()` — recognizes retention keywords
- `detect_retention_day()` — extracts D1/D3/D7/D14/D30 from query
- `compute_cohort_retention()` — calculates retention rate per cohort
- `compute_retention_summary()` — full D1/D3/D7/D14/D30 table
- `infer_cohort_date_column()` + `infer_activity_date_column()` — auto-detect cohort columns

Improvement 8: Single-Date CSV Support for Retention

Problem: Many CSVs only have one date column, but retention needs two. **Fix:** In `compute_cohort_retention()`, when `cohort_date_col == activity_date_col`:

```
user_first_date = work.groupby(user_col)[cohort_date_col].min().reset_index()
work = work.merge(user_first_date, on=user_col, how="left")
```

This derives cohort date as each user's first date automatically.

Improvement 9: Added Highest/Lowest Retention Detection

Problem: Could only find highest retention, not lowest. **Fix:** Added `detect_wants_lowest()` and `detect_wants_highest()` functions:

- "lowest d7 retention" → finds minimum rate
- "highest d7 retention" → finds maximum rate

- Message includes average and median across all cohorts

Improvement 10: Switched to Single-File Architecture

Problem: Recurring `ImportError` on Streamlit Cloud despite `sys.path` fixes. **Root Cause:** Streamlit Cloud's module resolution is inconsistent. **Fix:** Merged `query_engine.py` into `app.py` — no imports, no failures.

Improvement 11: Added Fuzzy Column Matching

Problem: Typos in column names or non-standard naming. **Fix:** Used `rapidfuzz.process.extractOne()` with `fuzz.token_sort_ratio` and 80% threshold to match user questions to column names.

Improvement 12: Added Error Handling & Diagnostics

Problem: Generic "No usable rows" errors with no context. **Fix:** Added detailed error messages showing:

- Which columns were used
- Sample raw values
- Sample parsed values
- Null counts after parsing

Why Deterministic

Cloud LLM Approach	Our Deterministic Approach
Calls OpenAI API	Fully offline
May hallucinate code	Same input = same output always
Requires internet	Works on airplane
Costs money per query	Free forever
Hard to debug	Easy to trace logic
Needs API keys	Zero credentials

This makes the app ideal for:

- Internal company analytics (data privacy)
- Airplane/offline demos
- Learning projects
- Predictable production pipelines

Limitations

1. **Column names matter:** Best results with descriptive names (`user_id`, `signup_date`). Obscure names may need manual selection.

2. **Structured queries only:** Free-form reasoning ("why did sales drop?") is not supported.
3. **Single CSV at a time:** No multi-file joins or database connections.
4. **English queries only:** No multilingual support yet.
5. **Retention needs sufficient data:** Cohorts with 1-2 users produce unreliable rates.

Future Roadmap

- Chart generation (line charts for DAU trends, bar charts for retention)
- Multi-column groupby (`profit by category and region`)
- Time-series trends (`sales trend by month`)
- Comparison queries (`sales this month vs last month`)
- Excel file support (`.xlsx`)
- Custom metric definitions via UI
- Export full PDF reports
- More retention days (D60, D90, D180)
- Cohort LTV (lifetime value) analysis

Example Test CSV

Save as `test_data.csv`:

```
user_id,signup_date,event_date,event_type,category,region,revenue,profit,plan
u1,2026-04-01,2026-04-01,signup,Electronics,North,0,0,basic
u1,2026-04-02,2026-04-02,login,Electronics,North,0,0,basic
u1,2026-04-08,2026-04-08,purchase,Electronics,North,120,36,premium
u2,2026-04-01,2026-04-01,signup,Furniture,South,0,0,free
u2,2026-04-03,2026-04-03,login,Furniture,South,0,0,free
u3,2026-04-01,2026-04-01,signup,Clothing,East,0,0,basic
u3,2026-04-08,2026-04-08,login,Clothing,East,0,0,basic
u4,2026-04-05,2026-04-05,signup,Electronics,West,0,0,premium
u4,2026-04-06,2026-04-06,login,Electronics,West,0,0,premium
u4,2026-04-12,2026-04-12,purchase,Electronics,West,200,60,premium
```

Test Questions:

- `how many rows`
- `profit by category`
- `highest d7 retention cohort`
- `dau last week`
- `average revenue by plan`

License

Summary

This project is a **fully local, deterministic CSV analytics assistant** that feels intelligent without using cloud AI. It combines Streamlit for UI, Pandas for analytics, RapidFuzz for fuzzy matching, and Python for logic — all running offline with predictable, debuggable behavior.

Key Achievement: Natural-language questions like "which cohort has highest d7 retention?" are translated into precise pandas operations through a deterministic rule engine, not an LLM.